

Assignment 1

CS3853, Computer Architecture

Reverse Engineering

Purpose

This assignment will help you understand the process of reverse engineering as well as deepen your understanding of x86 assembly programming.

Prerequisites to Review

You should understand the basic operation of the x86 instructions and addressing modes. This material was covered in lectures and in the tutorial, “A Tiny Guide to Programming in 32-bit x86 Assembly Language”.

You can find the x86 tutorial at <https://www.cs.virginia.edu/~cr4bd/4630/S2017/x86-doc.pdf>

Due Date

Assignment

Examine the following C code.

```
#include <stdio.h>
#define BUF_SIZE 13

int foo(){
    int i;
    int B[BUF_SIZE];
    for(i = 0; i < BUF_SIZE; i++)
        B[i] = 5;
    return i;
}

int main(){

    foo();

    return 0;
}
```

The assembly code produced for these two functions by gcc on Debian Stretch is the following (depending on the version of gcc being used). The code was produced using the following command:

```
$ gcc -m32 -c -S -masm=intel code.c
```

Depending on the system and compiler, additionally options, such as `-fno-asynchronous-unwind-tables` `-fno-pie` `-no-pie`, may also be required to simplify the output (disable PIE code and debug info).

The assembly file (`code.s`) is available on the Blackboard (use this assembly file—do not compile the code with your version of gcc).

```
1.      .file    "code.c"
2.      .intel_syntax noprefix
3.      .text
4.      .globl   foo
5.      .type    foo, @function
6. foo:
7.      push     ebp
8.      mov      ebp, esp
9.      sub      esp, 64
10.     mov      DWORD PTR [ebp-4], 0
11.     jmp      .L2
12.     .L3:
13.     mov      eax, DWORD PTR [ebp-4]
14.     mov      DWORD PTR [ebp-56+eax*4], 5
15.     add      DWORD PTR [ebp-4], 1
16.     .L2:
17.     cmp      DWORD PTR [ebp-4], 12
18.     jle      .L3
19.     mov      eax, DWORD PTR [ebp-4]
20.     leave
21.     ret
22.     .size     foo, .-foo
23.     .globl   main
24.     .type    main, @function
25. main:
26.     push     ebp
27.     mov      ebp, esp
28.     call     foo
29.     mov      eax, 0
30.     pop      ebp
31.     ret
32.     .size     main, .-main
33.     .ident    "GCC: (Debian 6.3.0-18+deb9u1) 6.3.0 20170516"
34.     .section      .note.GNU-stack,"",@progbits
```

Examine the source code and the assembly language. Relate the assembly code back to the source code. For example, can you identify the section of the assembly code that corresponds to the for loop in

function `foo`? After you have developed an understanding of the assembly code answer the following questions. When a question asks for the address of a variable, your answer should be of the form of the effective address of the variable. For example, `[ebp-4]`, `eax`.

1. Which lines of code correspond to the `for` loop of `foo`?
2. Which register is used to hold the variable `i` of `foo`?
3. What is the memory address of the variable `i` of `foo`?
4. What is the beginning address of array `B`?
5. What is the addressing mode for the constant `BUF_SIZE` when it is used as loop bound?
6. What is the addressing mode for the constant `5`?
7. What is the addressing mode used to access each element of `B` in the loop, i.e., `B[i]`?
8. What does the code on line 9 ("`sub esp, 64`") do?

Items to Submit

1. Submit the answers to Canvas.